

Future Additional Plan (Post v0.3.0)

Project Status: v0.3.0

Current completed milestones:

- ☒ Phase 1 - Minecraft ↔ Backend communication
- ☒ Phase 2 - Memory System & Tool Registry
- ☒ Phase 3 - Gemini LLM Planner

This document contains ideas intentionally postponed until the core AI assistant is stable. None of these features are immediate priorities. They are intended for future versions after the backend architecture and tool ecosystem mature.

Long-Term Vision

The project should evolve from:

"An AI chatbot inside Minecraft"

into

"A complete AI platform for Minecraft."

The AI backend should remain the core of the system.

The dashboard should become a management interface for the AI rather than the AI itself.

1. AI Dashboard

Objective

Develop a standalone dashboard that acts as the central control panel for the Minecraft AI Assistant.

The dashboard should eliminate the need for users to manually edit:

- `.env`
- `config.json`
- `memory.json`

Everything should be configurable through an intuitive interface.

Technology

Preferred stack:

- React
- TypeScript
- HTML
- CSS
- Vite

Reasoning:

The dashboard will eventually contain many independent pages and large amounts of state.

React is a better long-term choice than plain HTML/CSS/JavaScript.

2. Dashboard Theme

The interface should feel inspired by Minecraft.

Not an exact recreation.

Instead:

- Dark stone palette
- Creeper green accents
- Pixel-inspired icons
- Minecraft-inspired typography (optional)
- Redstone style status indicators
- Clean modern UI with Minecraft personality

Goal:

A user should immediately recognize that this dashboard belongs to the Minecraft ecosystem.

3. Dashboard Pages

Home

Overview of the AI system.

Display:

- Backend status
 - Gemini connection
 - Current model
 - Response latency
 - Number of memories
 - Installed tools
 - Version
 - Minecraft connection status
-

AI Configuration

Manage:

Provider

Model

Temperature

Max Tokens

Timeout

Debug Mode

Prompt Logging

Future provider support:

- Gemini
- OpenAI
- OpenRouter
- Ollama
- Anthropic

API keys should be stored securely.

No manual editing of .env files.

Memory Manager

Graphical interface for memory.

View:

Locations

Notes

Preferences

Features:

Search

Rename

Delete

Edit

Import

Export

Future:

Memory categories

Memory tags

Memory statistics

Tool Manager

Display every installed tool.

Information:

Tool Name

Description

Status

Version

Usage Statistics

Enable/Disable

Delete

Tool metadata should come directly from the backend registry.

Prompt Inspector

Developer page.

Display:

System Prompt

User Prompt

Memory Summary

Available Tools

Gemini Response

PlannerResult

Execution Results

Purpose:

Debugging

Learning

Development

Logs

Display:

Planner logs

Tool execution

Errors

Warnings

Latency

Filtering

Search

Export

Statistics

Dashboard metrics.

Examples:

Messages processed

Tool calls

Average latency

Memory count

Provider usage

Token usage

Most frequently used tools

Planner success rate

4. Tool Builder

This is intended to become one of the defining features of the project.

Goal:

Allow users to create tools without manually modifying the backend source code.

Philosophy

Users should describe capabilities.

The system should generate the implementation.

The dashboard should function as a Tool Wizard.

Tool Creation Wizard

Instead of writing code immediately, the dashboard asks structured questions.

Examples:

Tool Name

Description

When should the AI use this tool?

Required inputs

Expected outputs

Does it call:

- HTTP API
- Local executable
- Python function
- External service

Authentication method

Examples

AI-Assisted Tool Generation

After the user completes the wizard:

Gemini receives the specification.

Gemini generates:

- Tool implementation
- Metadata
- Input schema
- Documentation
- Registration boilerplate

The dashboard validates the generated files before installation.

The generated tool is then automatically registered with the Tool Registry.

No manual editing should be required.

5. Tool SDK

Internally, every generated tool should follow the existing BaseTool interface.

Generated automatically:

- Name
- Description
- Usage Examples
- Input Schema
- Execute Method

Maintain compatibility with the existing Tool Registry.

6. Extension Philosophy

The project should prioritize:

"Build your own tools"

instead of

"Download tools."

Community sharing can exist later, but user-generated tools should remain the primary philosophy.

Every user should be able to extend their assistant using their own Gemini API key.

7. Future Possibilities

Potential future modules:

Inventory Manager

World Explorer

Pathfinding

Block Placement

Building Assistant

Mining Assistant

Combat Assistant

World Analysis

Command Executor

Automation

These should all remain independent tools.

No core architectural changes should be required.

8. Release Timing

The dashboard should NOT be developed immediately.

Recommended milestone:

After the backend reaches feature stability.

Suggested version:

v0.6.0 - v0.8.0

Reason:

Avoid repeatedly updating the dashboard while the backend architecture is still evolving.

The dashboard should be built on top of a mature API.

9. Design Principles

The dashboard should remain a management application.

Responsibilities:

- Configure the assistant
- Inspect memory
- Create tools
- View logs
- Monitor status

The backend remains responsible for:

- Planning
- Tool execution
- Memory management
- AI communication

This separation should always be preserved.

Final Vision

The long-term objective is not simply to create an AI Minecraft mod.

The goal is to create a modular AI platform where Minecraft is the first supported environment.

The dashboard should become the central interface through which users configure, extend, inspect, and personalize their own AI assistant without directly modifying the underlying source code.

This feature should only be implemented once the backend architecture has stabilized sufficiently to avoid frequent breaking changes.